

Face Off

[Face Off — Module-by-Module Test Prompts](#)

Face Off — Module-by-Module Test Prompts

Run these **in order**, one at a time, after the master build prompt has been executed. Each prompt is self-contained — paste it into Claude Code as its own message. Do not move to the next prompt until the current one reports a clean pass. If a prompt surfaces failures, ask Claude Code to fix them and re-run that same prompt before continuing.

Test Prompt 1 — Architecture & lint compliance audit

```
Audit the entire lib/ directory against these rules and report every violation with file path and line number:
1. No file/class exceeds 300 lines.
2. No file outside core/theme/ contains a raw hex color or Color(0x...) literal — everything must route through the ThemeExtension.
3. No feature under lib/features/ imports another feature's internal files directly (only lib/core/ or that feature's own public exports).
4. Run `dart format --line-length=120 lib/ test/` and report whether it produced any diff.
5. Run `flutter analyze` and report every warning/error, not just errors.
Do not fix anything yet — just produce a clean audit report first.
```

Test Prompt 2 — Design system smoke tests

```
Write and run widget smoke tests for every component in core/widgets/: GradientScaffold, CoinBadge, PodiumLeaderboard, RoomCard, PinCodeEntry, ActivityToast, CollapsiblePanel, StatTile, DuelVsTransition, PrimaryPillButton, SecondaryPillButton. Each test should pump the widget with representative sample data and assert it renders without throwing, plus one meaningful interaction test where applicable (e.g., PinCodeEntry accepts input and reaches its "found" state; CollapsiblePanel toggles open/closed). Report pass/fail per widget.
```

Test Prompt 3 — App shell & navigation behavior

```
Verify the 3-tab shell: 1) each tab maintains its own back-stack when switching tabs and returning, 2) the floating bottom nav hides on scroll-down past the threshold and reappears on scroll-up, 3) it also reappears after ~600ms of scroll idle, 4) it never hides while a modal or bottom sheet is open. Write integration/widget tests covering these four behaviors specifically. Report pass/fail for each.
```

Test Prompt 4 — Auth & onboarding flow

```
Test the full first-launch flow end to end using mocked Firebase Auth: 1) Lottie welcome sequence displays and advances correctly, 2) Google sign-in path creates a Firestore profile document with correct default values on first sign-in, 3) Apple sign-in does the same, 4) a returning user (already has a profile doc) skips straight to the app shell, 5) the showcaseview onboarding sequence runs exactly once and never replays on a second app launch (test this against local storage state, not just in-memory state). Report pass/fail per step, and flag anything that required real network access instead of mocks.
```

Test Prompt 5 — Matchmaking / Play tab

```
Test the matchmaking flow with a mocked queue backend: 1) tapping Quick Match enters the queue state with a visible, working cancel button, 2) two mocked waiting users get paired and both receive the same match ID, 3) the timeout path (no opponent found within the configured window) shows the retry/cancel prompt correctly, 4) cancelling from the queue correctly removes the user from the queue (no ghost queue entries). Report pass/fail per step.
```

Test Prompt 6 — Duel state machine (this is the most important test file — take it seriously)

Using the fake/mock signaling layer built in Section 8.7 of the build prompt, write and run unit tests for the round state machine covering every one of these scenarios explicitly, each as its own test case:

1. Normal exchange: player A fires after cue, player B does not dodge in time – A wins the round.
2. Successful dodge: player A fires after cue, player B dodges within the reaction window – verify the round continues per the rule as specified in the build prompt (not an instant win for B), and confirm the alternate-rule TODO flag exists and is toggleable.
3. False start: player A fires before the cue timestamp – A instantly loses, verified against the server timestamp not local time.
4. Crack during Neutral phase – that player loses immediately, before any cue has armed.
5. Crack during CueArmed phase (before cue fires) – same result.
6. Crack during CueFired phase – same result.
7. Simultaneous crack within the jitter window (test at exactly the boundary, e.g., 149ms and 151ms apart) – 149ms should draw, 151ms should NOT draw (verify the boundary is handled correctly, not off-by-one).
8. Round timeout: neither player fires within the timeout window – draw, and confirm a diagnostic event is logged.
9. Full match: simulate a complete best-of-5 sequence ending 3-1 and confirm MatchResult fires with correct final score and winner.
10. Disconnect mid-round: simulate one client going offline, confirm the reconnect grace period triggers and either resolves correctly on reconnect or forfeits gracefully after the timeout.

Report pass/fail for each of the 10 scenarios individually, not as one aggregate pass/fail.

Test Prompt 7 — Gesture engine isolation

Verify the gesture engine's isolate boundary is real, not accidental: 1) confirm the blendshape thresholding math actually executes on a separate isolate (not just wrapped in an async function on the main isolate), 2) confirm the duel feature's domain layer only ever receives the semantic events (fireDetected, dodgeDetected, crackDetected) and never touches raw blendshape values directly, 3) write a test that feeds fake blendshape data through the pipeline and asserts the correct semantic event is emitted at the correct threshold boundaries. Report pass/fail and flag any place raw blendshape data leaks outside core/gesture_engine/.

Test Prompt 8 — Friends feature

Test: 1) sending a friend request via invite link and via PIN code both correctly create a pending request record, 2) accepting/declining updates state correctly on both sides, 3) blocking a user immediately prevents that user from appearing in future matchmaking pairing – test this specifically against the server-side/security-rule enforcement, not just the client-side hide, 4) reporting a user requires a reason selection and correctly writes to the reports collection, 5) unfriending removes the relationship symmetrically for both users. Report pass/fail per step.

Test Prompt 9 — Profile & subscription management

Test: 1) the stat grid correctly reflects match history data (win streak, total matches, friends count, win rate) against a seeded mock dataset, 2) the leaderboard component correctly ranks and displays a seeded set of mock users, 3) the subscription section correctly reflects Free vs Face Off Plus state from a mocked RevenueCat customer info object, 4) Restore Purchases triggers the correct RevenueCat call and updates UI state on success, 5) account deletion actually removes both the Firestore profile and the Firebase Auth account (test against a mock, but assert both deletion calls are made, not just one).

Test Prompt 10 — Monetization & ad placement

Test: 1) the cosmetics store correctly fetches and displays a mocked RevenueCat Offerings response, 2) purchasing a cosmetic updates local ownership state and equips it correctly, 3) the Face Off Plus paywall triggers correctly from both the Profile entry point and a gated feature touchpoint (e.g., HD export), 4) the lobby ad banner tile renders in the correct position on the Play tab without shifting other layout elements, 5) the post-match rewarded ad offer is opt-in only – write a test that asserts no ad-related SDK call fires unless the user explicitly taps the offer, 6) confirm no ad placement exists anywhere inside the duel feature's active-match screens by searching for any ad-related widget or SDK call within lib/features/duel/.

Test Prompt 11 — Offline handling

Test: 1) simulating a connectivity loss correctly triggers the bottom sheet within a reasonable delay, 2) the sheet auto-dismisses when connectivity is restored and the server health check passes, without requiring manual dismissal, 3) manual swipe-down dismissal still works independently, 4) triggering a connectivity loss during an active duel match correctly pauses the round and shows the reconnect-grace-period state to the opponent, 5) exceeding the reconnect grace period correctly forfeits the match gracefully rather than hanging. Report pass/fail per step.

Test Prompt 12 — Full regression + performance pass

Run the entire existing test suite (all prior prompts' tests) together and confirm nothing regressed. Then: 1) run flutter analyze one more time and confirm zero warnings/errors, 2) run dart format --line-length=120 --set-exit-if-changed lib/ test/ and confirm exit code 0, 3) re-run the 300-line-file audit from Test Prompt 1 and confirm still zero violations, 4) report current widget rebuild hotspots using DevTools if available in this environment, or flag which subtrees you were unable to profile in this environment and what a human should check manually on a real device before submission.